

[DOC] prueba

Documentación Técnica · AutoDocs AI · 14/07/2026

8	10	1	4
Archivos	Funciones	Clases	Endpoints

Score de Calidad: 82/100

Documentación del Proyecto

Generado automáticamente por **AutoDocs AI** — 2026-07-14 03:16 UTC

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **Python** y cuenta con 8 archivos fuente, 10 funciones, 1 clases y 4 endpoints de API documentados. El score de calidad del código es **82/100**.

Stack Tecnológico

Lenguaje	Cantidad
Python	8 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	8
Funciones	10
Clases	1
Endpoints API	4
Score de calidad	82/100
Funciones documentadas	4 / 10
Clases documentadas	0 / 1

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/  
  ■■■ (raíz)/  
  ■■■ controllers/  
  ■■■ models/  
  ■■■ routes/
```

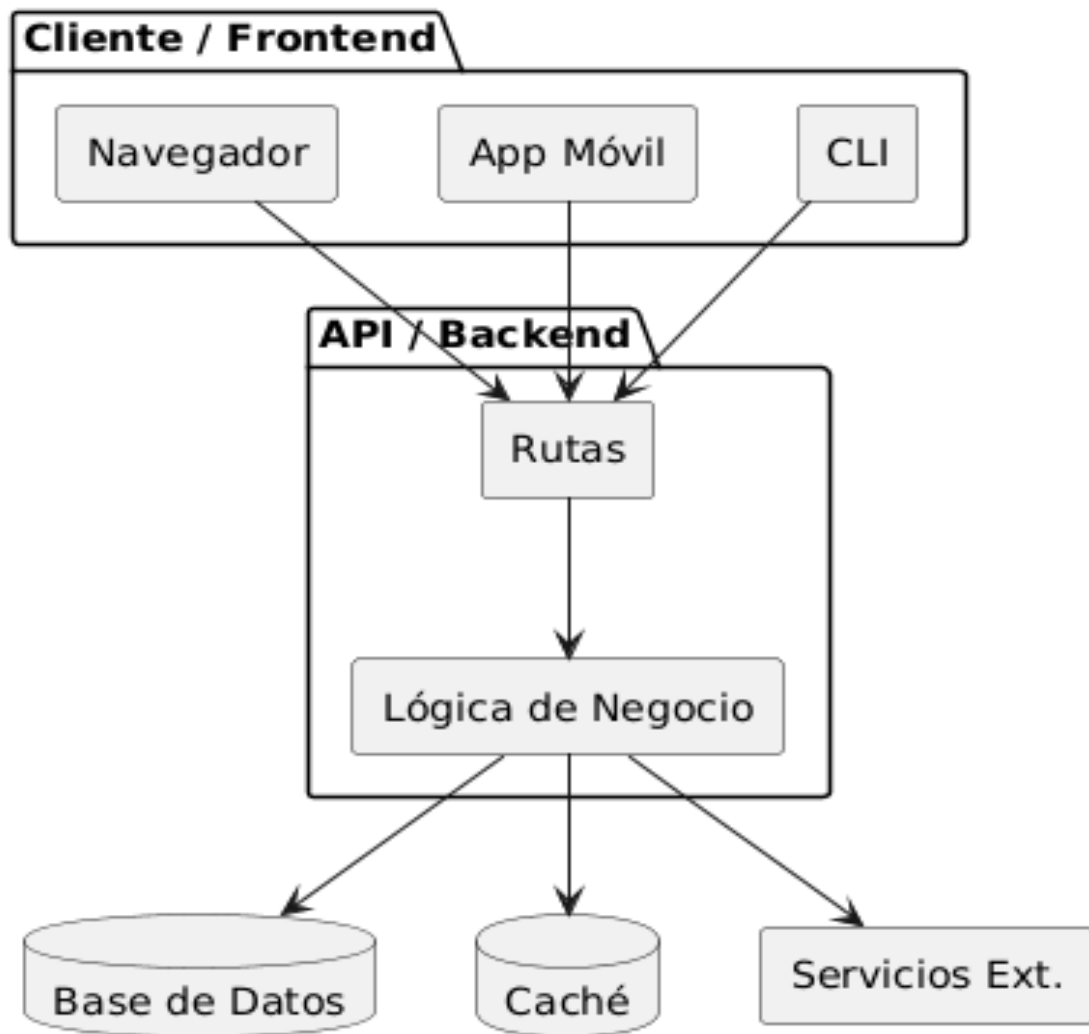
2.2 Distribución por Capas

- (raíz)/ — 6 archivos [.env(1), .md(1), .py(2), .gitignore(1), .txt(1)]
- controllers/ — 2 archivos [.py(2)]
- models/ — 2 archivos [.py(2)]
- routes/ — 2 archivos [.py(2)]

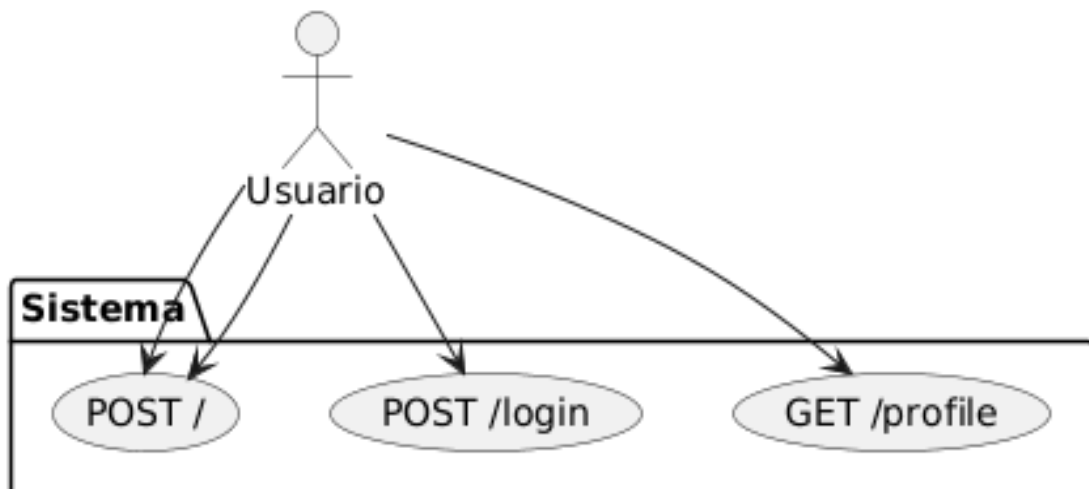
2.3 Distribución de Lenguajes

- Python: 8 archivos (100%)

2.4 Diagrama de Infraestructura



2.5 Diagrama de Casos de Uso



2.6 Diagrama de Clases



2.7 Patrones Detectados

- **API REST:** 4 endpoints detectados (Flask)
- **Programación Orientada a Objetos:** constructores detectados

2.8 Insights de Arquitectura (IA)

Análisis del Proyecto

- **Patrones arquitectónicos probables:** Monolito, debido a la presencia de un solo lenguaje y pocas clases.
- **Buenas prácticas detectadas:** Uso de un lenguaje principal (Python) y una cantidad moderada de funciones y endpoints.
- **Áreas de mejora sugeridas:** Incrementar la modularidad con más clases y considerar la separación de concerns (MVC).
- **Complejidad del proyecto:** Baja-Moderada, debido a la cantidad limitada de funciones y clases.
- **Recomendaciones de escalabilidad:** Considerar la adopción de patrones de diseño como MVC o Microservicios para permitir una mayor flexibilidad y escalabilidad a medida que el proyecto crece.

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate   # Windows

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Configurar variables de entorno
```

```
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: 4

Módulo: `routes/user.py`

[GET] GET /

- **Framework:** Flask
- **Archivo:** `routes/user.py`

Descripción del Endpoint

1. Propósito del endpoint:

El propósito de este endpoint es proporcionar una forma sencilla de obtener información sobre los usuarios en un sistema. Piensa en ello como una puerta de entrada para acceder a datos importantes.

2. Funcionalidad:

Este endpoint se utiliza para recopilar información sobre los usuarios. Cuando se accede a esta ruta, el sistema busca y recopila los datos de los usuarios y los devuelve en un formato que es fácil de entender.

3. Información necesaria:

Para que este endpoint funcione, no se necesita proporcionar mucha información. Simplemente necesitas acceder a la ruta correcta (/) y el sistema se encargará del resto. Es como visitar una página web: solo necesitas saber la dirección y listo.

4. Resultado:

El resultado que devuelve este endpoint es una lista de información sobre los usuarios. Esto puede incluir nombres, direcciones de correo electrónico, o cualquier otra información relevante que el sistema tenga almacenada sobre los usuarios. Piensa en ello como una hoja de cálculo con todos los datos de los usuarios.

5. Uso en la vida real:

En la vida real, este tipo de endpoint es muy útil en situaciones como:

- **Gestión de clientes:** Una empresa puede utilizar este endpoint para obtener una lista de todos sus clientes y sus detalles

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
- [400](#) — Solicitud incorrecta
- [401](#) — No autorizado
- [404](#) — Recurso no encontrado
- [500](#) — Error interno del servidor

[POST] [POST](#) /

- **Framework:** Flask
- **Archivo:** [routes/user.py](#)

Descripción del Endpoint de API

Propósito:

Este endpoint es como un mensajero que ayuda a crear algo nuevo en un sistema. En este caso, se utiliza para crear un nuevo usuario en un proyecto de Python.

Funcionalidad:

Cuando se envía una solicitud a este endpoint, lo que hace es recibir la información necesaria para crear un nuevo usuario y luego la guarda en la base de datos del proyecto. Es como llenar un formulario con tus datos y enviarlo para que te creen una cuenta en un sitio web.

Información necesaria:

Para que este endpoint funcione, necesita recibir algunos datos importantes como, por ejemplo, el nombre del usuario, la contraseña y tal vez el correo electrónico. Es como cuando te registras en una página web y te piden algunos datos personales.

Resultado:

Una vez que el endpoint recibe la información y la procesa, devuelve un mensaje que indica si la creación del usuario fue exitosa o no. Si todo salió bien, te dirá que el usuario fue creado con éxito. Si hubo un problema, te dirá qué fue lo que salió mal.

Uso en la vida real:

En la vida real, este tipo de endpoint se utiliza en sitios web y aplicaciones móviles cuando necesitas crear una cuenta para acceder a ciertos servicios. Por ejemplo, cuando te registras en una red social, un sitio de compr

Parámetros esperados:

- [body](#) (JSON, requerido): Cuerpo de la solicitud
- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
- [400](#) — Solicitud incorrecta
- [401](#) — No autorizado

- [404](#) — Recurso no encontrado
- [500](#) — Error interno del servidor

[POST] [POST](#) /login

- **Framework:** Flask
- **Archivo:** [routes/user.py](#)

Descripción del Endpoint de API: /login

1. Propósito del endpoint:

El propósito de este endpoint es permitir a los usuarios acceder a un sistema o aplicación de manera segura. En otras palabras, es como la puerta de entrada a un sitio web o aplicación donde necesitas identificarte para poder usar sus funciones.

2. Funcionalidad:

Cuando se utiliza este endpoint, se envían tus credenciales de acceso (como tu nombre de usuario y contraseña) al servidor. Luego, el servidor verifica si esas credenciales son correctas y, si lo son, te permite acceder al sistema o aplicación. Es como introducir tu llave en la cerradura de una puerta: si la llave es la correcta, la puerta se abre.

3. Información necesaria:

Para que este endpoint funcione, necesitas proporcionar dos piezas de información básica:

- **Nombre de usuario:** Es el nombre que usas para identificarte en el sistema.
- **Contraseña:** Es la palabra secreta que solo tú conoces y que se utiliza para verificar tu identidad.

4. Resultado:

Si las credenciales que proporcionas son correctas, el endpoint te devuelve un resultado que indica que has accedido exitosamente. Esto puede incluir un mensaje de bienvenida, un token de acceso (una especie de llave digital)

Parámetros esperados:

- [body](#) (JSON, requerido): Cuerpo de la solicitud
- [Authorization](#) (Header, requerido): Bearer JWT token

Códigos de respuesta:

- [200](#) — Operación exitosa
- [400](#) — Solicitud incorrecta
- [401](#) — No autorizado
- [404](#) — Recurso no encontrado
- [500](#) — Error interno del servidor

[GET] [GET](#) /profile

- **Framework:** Flask
- **Archivo:** `routes/user.py`

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa
 - **400** — Solicitud incorrecta
 - **401** — No autorizado
 - **404** — Recurso no encontrado
 - **500** — Error interno del servidor
-

3.3 Estado de Comentarios y Docstrings

Total de funciones: 10

- **[OK] Documentadas:** 4 (40%)
- **[X] Sin documentar:** 6 (60%)

Visualización de documentación:

[██] 40% documentado

Ejemplo de docstrings detectados:

```
def index():
    """
    Obtener todos los usuarios
    ---
    tags:
      - Usuarios
    responses:
      200:
        description: Lista de usuarios
    """
    ...
```

```
def user_store():
    """
    Crear un usuarios
    ---
    tags:
      - Usuarios
    parameters:
      - in: body
        name: user
        schema:
          type: object
```

```

    required:
      - name
      - email
      - password
    properties:
      name:
        type: string
        example: "Juan Pérez"
      email:
        type: string
        example: "juan@example.com"
      password:
        type: string
        example: "123456"
  responses:
    201:
      description: Usuario creado
      ""
      ...

```

```

def login():
    """
    Iniciar sesión
    """
    ---
    tags:
      - Autenticación
    parameters:
      - in: body
        name: credentials
        schema:
          type: object
          required:
            - email
            - password
          properties:
            email:
              type: string
              example: "juan@example.com"
            password:
              type: string
              example: "123456"
    responses:
      200:
        description: Login exitoso
        ""
        ...

```

Funciones que requieren documentación:

- `get_all_users()` en `controllers/userController.py`
- `create_user(name, email, password)` en `controllers/userController.py`
- `login_user(email, password)` en `controllers/userController.py`
- `__init__(self, name, email, password)` en `models/User.py`
- `check_password(self, password)` en `models/User.py`

- ... y 1 más

3.4 Planes de Prueba (Testing)

No se detectaron pruebas automatizadas. Se recomienda implementar:

```
# Ejemplo de estructura de tests recomendada
import pytest

class TestEndpoints:
    def test_get_returns_200(self, client):
        response = client.get('/api/endpoint')
        assert response.status_code == 200

    def test_post_creates_resource(self, client):
        data = {'name': 'test'}
        response = client.post('/api/endpoint', json=data)
        assert response.status_code == 201

    def test_unauthorized_returns_401(self, client):
        response = client.get('/api/protected')
        assert response.status_code == 401
```

Escenarios de prueba recomendados:

Escenario	Tipo	Prioridad
GET / — respuesta 200	Integración	Alta
GET / — sin auth (401)	Seguridad	Alta
POST / — respuesta 200	Integración	Alta
POST / — sin auth (401)	Seguridad	Alta
POST /login — respuesta 200	Integración	Alta
POST /login — sin auth (401)	Seguridad	Alta
GET /profile — respuesta 200	Integración	Alta
GET /profile — sin auth (401)	Seguridad	Alta

4. Documentación de Clases

Total: **1** clases

User

- **Archivo:** `models/User.py`
- **Línea:** 4
- **Hereda de:** `db.Model`

- **Métodos:** 3
- **Documentada:** [X] No

Introducción a la Clase "User"

La clase "User" es como un archivo personal en un sistema de gestión de información. Su propósito es almacenar y gestionar la información de los usuarios que interactúan con un sistema o aplicación.

Qué representa la Clase "User"

Imagina que estás en una biblioteca y necesitas una tarjeta de membresía para pedir prestados libros. La tarjeta de membresía tiene tu nombre, dirección, número de teléfono y otros datos personales. De la misma manera, la clase "User" es como una tarjeta de membresía virtual que almacena la información de los usuarios en un sistema.

Funcionalidades de la Clase "User"

La clase "User" puede hacer varias cosas:

- Almacenar información de los usuarios, como nombres, correos electrónicos y contraseñas.
- Verificar si una contraseña es correcta (mediante el método "check_password").
- Convertir la información del usuario en un formato fácil de leer y entender (mediante el método "to_dict").

Uso en la vida real

La clase "User" se utiliza en muchas aplicaciones y sistemas en línea, como:

- Sitios web de comercio electrónico, donde se almacena la información de los clientes para procesar pedidos y enviar correos electrónicos.
- Redes sociales, donde se almacena la información

Métodos:

- `__init__()`
 - `check_password()`
 - `to_dict()`
-

5. Documentación de Funciones

Total: **10 funciones**

Módulo: `controllers/userController.py`

`get_all_users`

- **Parámetros:** —
- **Async:** —

- Complejidad: [LOW] 4
- Documentada: [X]

La función "get_all_users" sirve para obtener una lista de todos los usuarios registrados en un sistema. Necesita acceso a la base de datos para funcionar. Produce una lista con información de cada usuario. En la vida real, se usa en sitios web para administrar cuentas de usuarios. Es importante porque permite a los administradores ver y gestionar todos los usuarios del sistema de manera eficiente.

`create_user`

- Parámetros: `name`, `email`, `password`
- Async: —
- Complejidad: [LOW] 3
- Documentada: [X]

La función "create_user" crea un nuevo usuario en el sistema. Necesita saber el nombre, correo electrónico y contraseña del usuario. Con esta información, crea una cuenta nueva y la guarda en la base de datos. En la vida real, se usa para que las personas puedan registrarse en una plataforma o sitio web. Es importante porque permite a los usuarios acceder a servicios y funcionalidades exclusivas. En resumen, esta función es fundamental para el registro de usuarios en el sistema, permitiendo una experiencia personalizada y segura.

`login_user`

- Parámetros: `email`, `password`
- Async: —
- Complejidad: [LOW] 3
- Documentada: [X]

La función "login_user" permite a los usuarios acceder a un sistema con su cuenta.

Necesita el correo electrónico y la contraseña para funcionar.

Una vez ingresados, verifica si los datos son correctos y permite el acceso.

Se utiliza en situaciones como iniciar sesión en una plataforma en línea o en un sitio web.

Es fundamental para la seguridad y el control de acceso en el sistema, garantizando que solo los usuarios autorizados puedan entrar.

Módulo: `models/User.py`

`__init__`

- Parámetros: `self`, `name`, `email`, `password`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función `__init__` crea un nuevo usuario. Necesita saber el nombre, correo electrónico y contraseña del usuario. Cuando se ejecuta, crea un nuevo perfil de usuario. En la vida real, se usa para

registrarse en sitios web o aplicaciones. Es importante porque permite a los usuarios acceder a servicios personalizados.

`check_password`

- Parámetros: `self`, `password`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "check_password" verifica si una contraseña es correcta. Necesita saber la contraseña que se quiere comprobar. Luego, dice si es correcta o no. Se usa en situaciones como cuando intentas iniciar sesión en una cuenta en línea. Esta función es importante porque ayuda a mantener seguras las cuentas de los usuarios, asegurando que solo los dueños de la cuenta puedan acceder a ella.

`to_dict`

- Parámetros: `self`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

Módulo: `routes/user.py`

`index`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

Obtener todos los usuarios

tags:

- Usuarios

responses:

200:

description: Lista de usuarios

`user_store`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

Crear un usuarios

tags:

- Usuarios

parameters:

- in: body

name: user

schema:

type: object

required:

- name

- email

- password

properties:

name:

type: string

example: "Juan Pérez"

email:

type: string

example: "juan@example.com"

password:

type: string

example: "123456"

responses:

201:

description: Usuario creado

[login](#)

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [OK]

Iniciar sesión

tags:

- Autenticación

parameters:

- in: body
name: credentials
schema:
type: object
required:
- email
- password
properties:
email:
type: string
example: "juan@example.com"
password:
type: string
example: "123456"
responses:
200:
description: Login exitoso

profile

- Parámetros: —
- Async: —
- Complejidad: [LOW] 2
- Documentada: [OK]

Obtener perfil del usuario autenticado

tags:
- Usuarios
responses:
200:
description: Datos del usuario

index()

Ubicación: `routes/user.py`

Obtener todos los usuarios

tags:
- Usuarios

responses:

200:

description: Lista de usuarios

`user_store()`

Ubicación: `routes/user.py`

Crear un usuarios

tags:

- Usuarios

parameters:

- in: body

name: user

schema:

type: object

required:

- name

- email

- password

properties:

name:

type: string

example: "Juan Pérez"

email:

type: string

example: "juan@example.com"

password:

type: string

example: "123456"

responses:

201:

description: Usuario creado

`login()`

Ubicación: `routes/user.py`

Iniciar sesión

tags:

- Autenticación

parameters:

- in: body

name: credentials

schema:

type: object

required:

- email

- password

properties:

email:

type: string

example: "juan@example.com"

password:

type: string

example: "123456"

responses:

200:

description: Login exitoso

`profile()`

Ubicación: `routes/user.py`

Obtener perfil del usuario autenticado

tags:

- Usuarios

responses:

200:

description: Datos del usuario

6. Modelos de Datos

No se detectaron clases con nombres de modelo explícitos.

Esquema General Inferido

Basado en el análisis del código, las entidades principales son:

- User
 - All Users
-

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

- Lenguaje principal: Python
- Archivos detectados: 8

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
python app.py
```

7.3 Despliegue con Docker

```
FROM python:3.12-slim

WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8000
CMD ["python app.py"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
```

```
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

- ☐ Variables de entorno configuradas
- ☐ Base de datos migrada
- ☐ Tests ejecutados y pasando
- ☐ Logs configurados
- ☐ HTTPS habilitado en producción
- ☐ Backups programados
- ☐ Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-07-14

- Versión inicial analizada por AutoDocs AI
- Documentación técnica generada automáticamente
- Análisis de calidad del código completado

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 82/100 (Excelente [GREEN])

Distribución de Documentación:

- Funciones documentadas: 4/10 (40%)
- Clases documentadas: 0/1

Análisis de Complejidad:

- Complejidad promedio: 1.8

- Complejidad máxima: 4
- Funciones con alta complejidad (>10): 0

Tendencia de Complejidad por Archivo:

[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: ↓ Baja

8.2 Funciones con Alta Complejidad Ciclomática

No hay funciones con complejidad > 10. [OK]

8.3 Recomendaciones

- Agregar docstrings a las 6 funciones sin documentar

8.4 Errores de Análisis

Sin errores detectados. [OK]

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

- **Funciones sin documentar:** 6

- `get_all_users` en `controllers/userController.py`
- `create_user` en `controllers/userController.py`
- `login_user` en `controllers/userController.py`
- `__init__` en `models/User.py`
- `check_password` en `models/User.py`

- ... y 1 más

9.4 Minería de Datos Aplicada al Código

Análisis automático realizado sobre las funciones y clases extraídas del proyecto.

A) Funciones de Alta Complejidad Ciclomática

Nivel	Umbral	Cantidad	Descripción
■ Alta	> 5	0	Requieren refactorización urgente
■ Media	4–5	1	Candidatas a simplificación
■ Baja	≤ 3	9	Complejidad aceptable

■ No se detectaron funciones con complejidad ciclomática alta. El código mantiene un buen nivel de simplicidad.

B) Funciones Sin Documentación

6 de 10 funciones carecen de docstring (60%).

Función	Archivo	Línea	Complejidad
`get_all_users`	`controllers/userController.py`	6	4
`create_user`	`controllers/userController.py`	13	3
`login_user`	`controllers/userController.py`	23	3
`__init__`	`models/User.py`	10	1
`check_password`	`models/User.py`	15	1
`to_dict`	`models/User.py`	18	1

■ **Sugerencia:** Agrega docstrings a todas las funciones públicas. Prioriza las de mayor complejidad primero, ya que son las más difíciles de entender sin documentación.

C) Detección de Funciones con Nombres Similares (Posibles Duplicados)

Se comparan los nombres de funciones entre archivos para detectar lógica duplicada o que podría unificarse.

■ No se detectaron funciones con nombres similares en archivos distintos.

D) Análisis de Módulos — Archivos con Mayor Concentración de Funciones

Archivo	Funciones	Complejidad Promedio	Documentadas	Estado
`routes/user.py`	4	1.2	100%	■ OK
`controllers/userController.py`	3	3.3	0%	■ OK
`models/User.py`	3	1.0	0%	■ OK

E) Patrones Frecuentes Adicionales (Análisis IA)

- **init**: 2 ocurrencias

- Ejemplos: ['__init__', 'init']

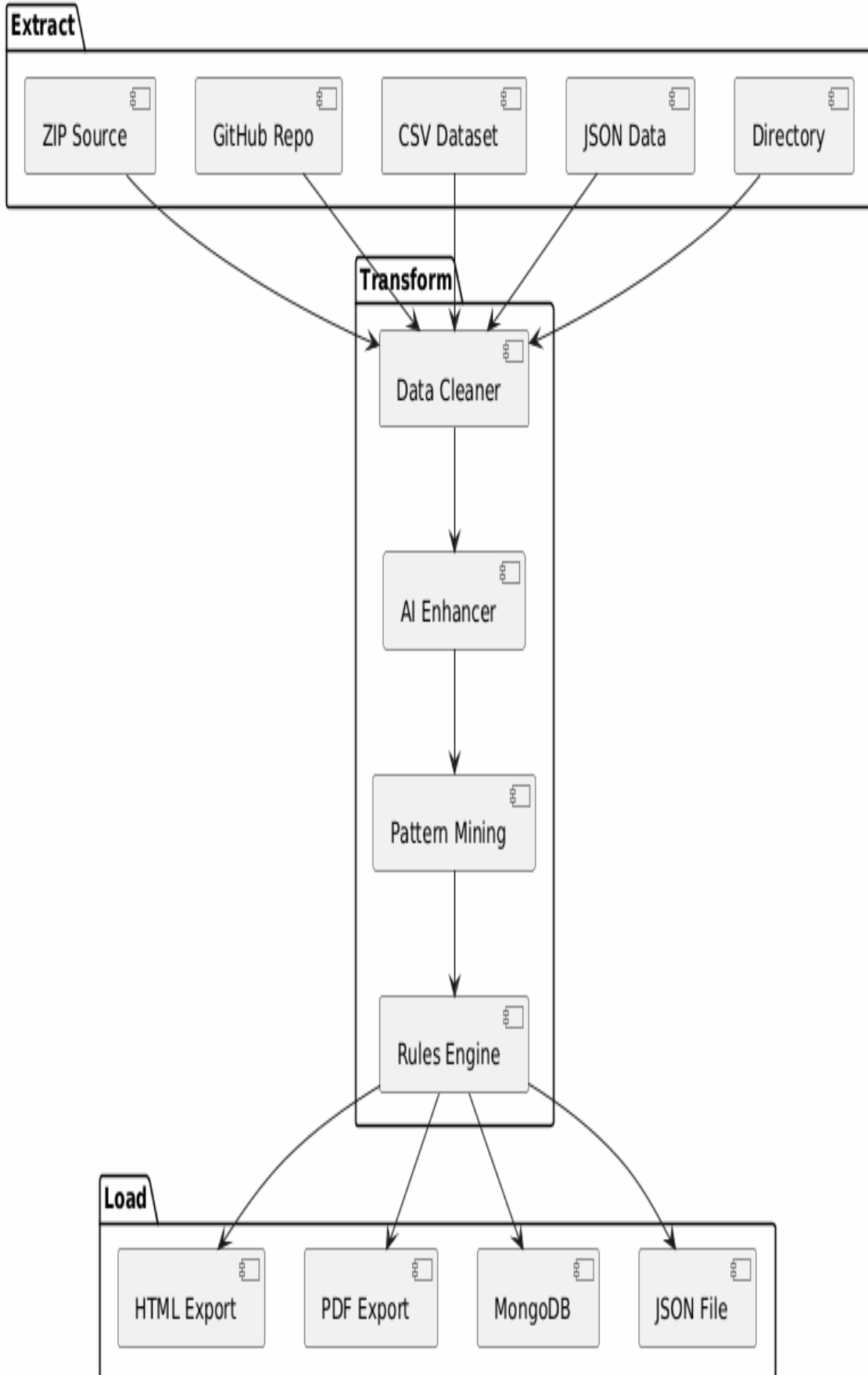
- **user**: 5 ocurrencias

- Ejemplos: ['create_user', 'login_user', 'get_all_users', 'User', 'user_store']

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):

Pipeline ETL - AutoDocs AI



Fases del Pipeline:

- **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales
- **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones
- **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-07-14 03:16 UTC

Generado automáticamente por AutoDocs AI